Response Codes: How an Agent Knows What Just Happened

Every LLM reply ends with a response code: the model's reason for stopping. A single agent reads it after every call to decide what to do next, and every provider names it differently.

One reply from a single agent (Groq · OpenAI format)

```
{
  "model": "openai/gpt-oss-120b",
  "choices": [{
    "message": {
      "role": "assistant",
      "content": null,
      "tool_calls": [{
        "function": {
          "name": "run_sql",
          "arguments": "{\"ticket\": \"4521\"}"
        }
      }]
    },
    "finish_reason": "tool_calls"        ← response code
  }],
  "usage": {"prompt_tokens": 812, "completion_tokens": 41}
}
```

`content` is empty because the model wants a tool. The response code tells the loop what to do next.

Same idea, different names

What happened	Groq · OpenAI	Anthropic	Gemini
Field name	`finish_reason`	`stop_reason`	`finishReason`
Finished the answer	`stop`	`end_turn`	`STOP`
Wants a tool	`tool_calls`	`tool_use`	`STOP *`
Cut off (token limit)	`length`	`max_tokens`	`MAX_TOKENS`
Blocked or refused	`content_filter`	`refusal`	`SAFETY`

** Gemini says STOP even when it wants a tool; the function call sits in the reply's content.*

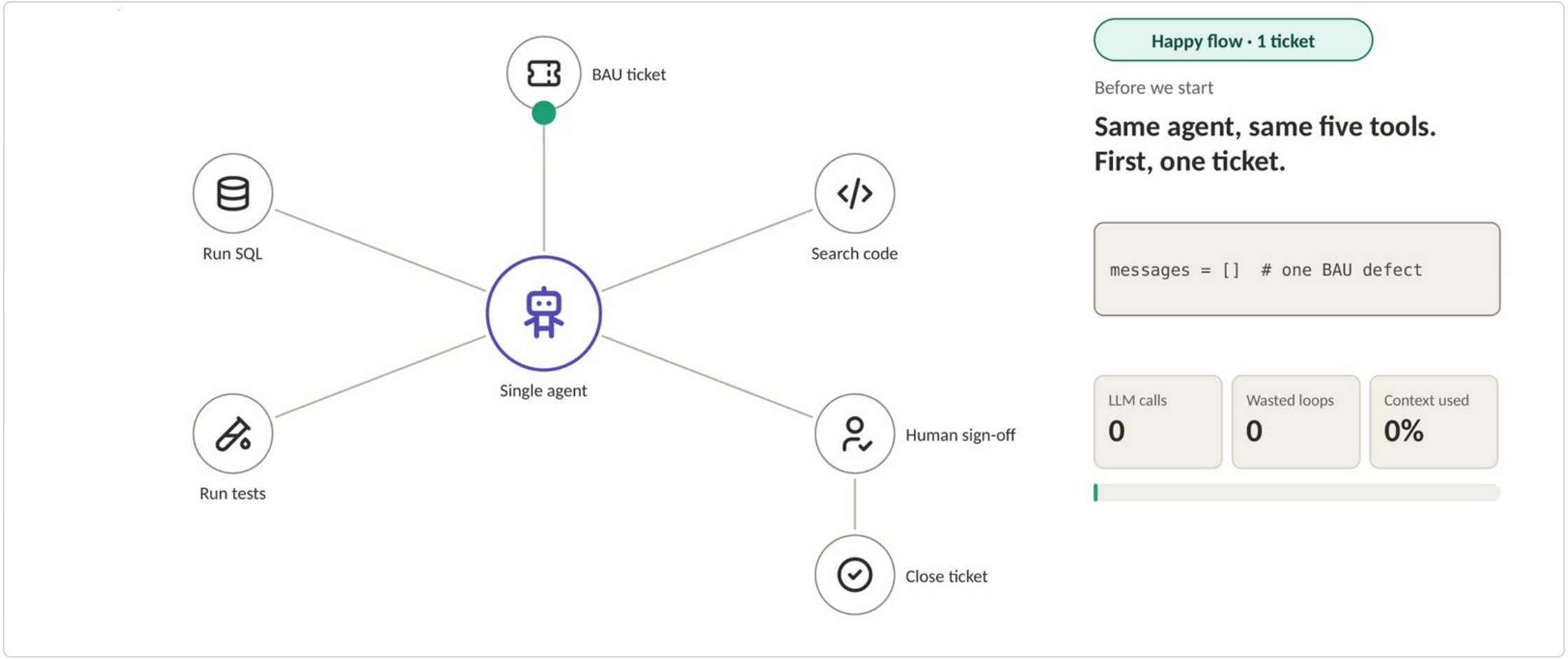
What the loop does next

- **`tool_calls`** → run the tool, call again ↻
- **`stop`** → return the answer, done
- **`length`** → raise the limit or continue
- **`content_filter`** → stop and escalate

One field steers the whole loop. Fine for one task, fragile at scale: and that's exactly where single agents start to fail →

Why Single Agents Fail · One Ticket

Agents suffer from overload too. With one BAU ticket, the agent stays focused: right query, right fix, sign-off.

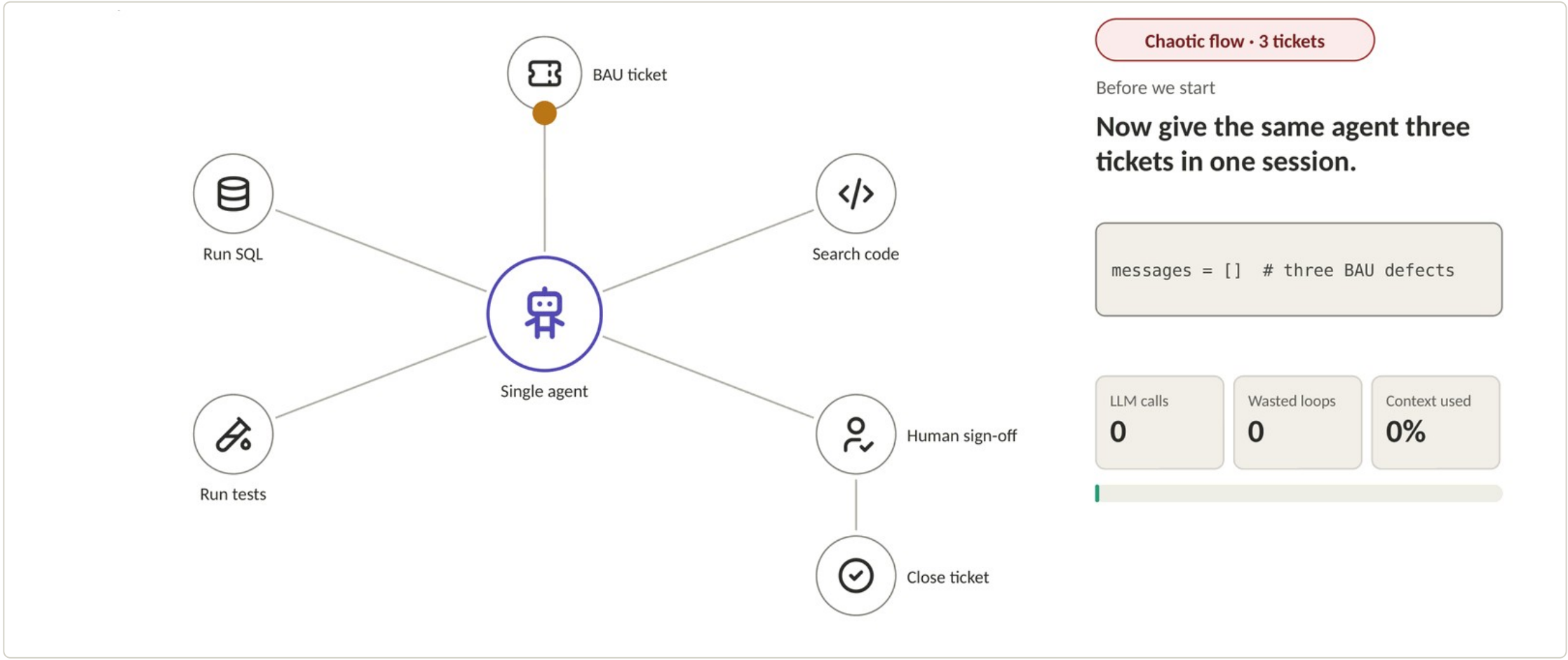


▶ Play animation

Same agent, same five tools: one ticket, fixed and signed off in 4 LLM calls.

Why Single Agents Fail · Three Tickets

Give the same agent three tickets in one session and its decisions get weaker: mixed-up data, repeated retries, a test edited until it passes.



▶ Play animation

A single agent is a monolith: it works for one task and fails like a monolith when you scale the work.

Why Single Agents Fail – Part 2

Thinking, acting and checking are mixed together in one loop. Sort the same messages by job and three separate responsibilities appear.

One agent, one context window · time flows down ↓

Read 3 tickets	`no_signal`
SQL: SELECT *	`tool_use`
50k rows	`tool`
Suspect join	`no_signal`
Search code	`tool_use`
Switch to 4522	`no_signal`
SQL: 4522	`tool_use`
Test 4521 + 4522 data	`tool_use`
Fail	`tool`
Retry same fix	`tool_use`
Fail again	`tool`
Edit the test	`tool_use`
False pass	`tool`
Ask sign-off	`end_turn`
Human rejects	`user`
Context full	`max_tokens`

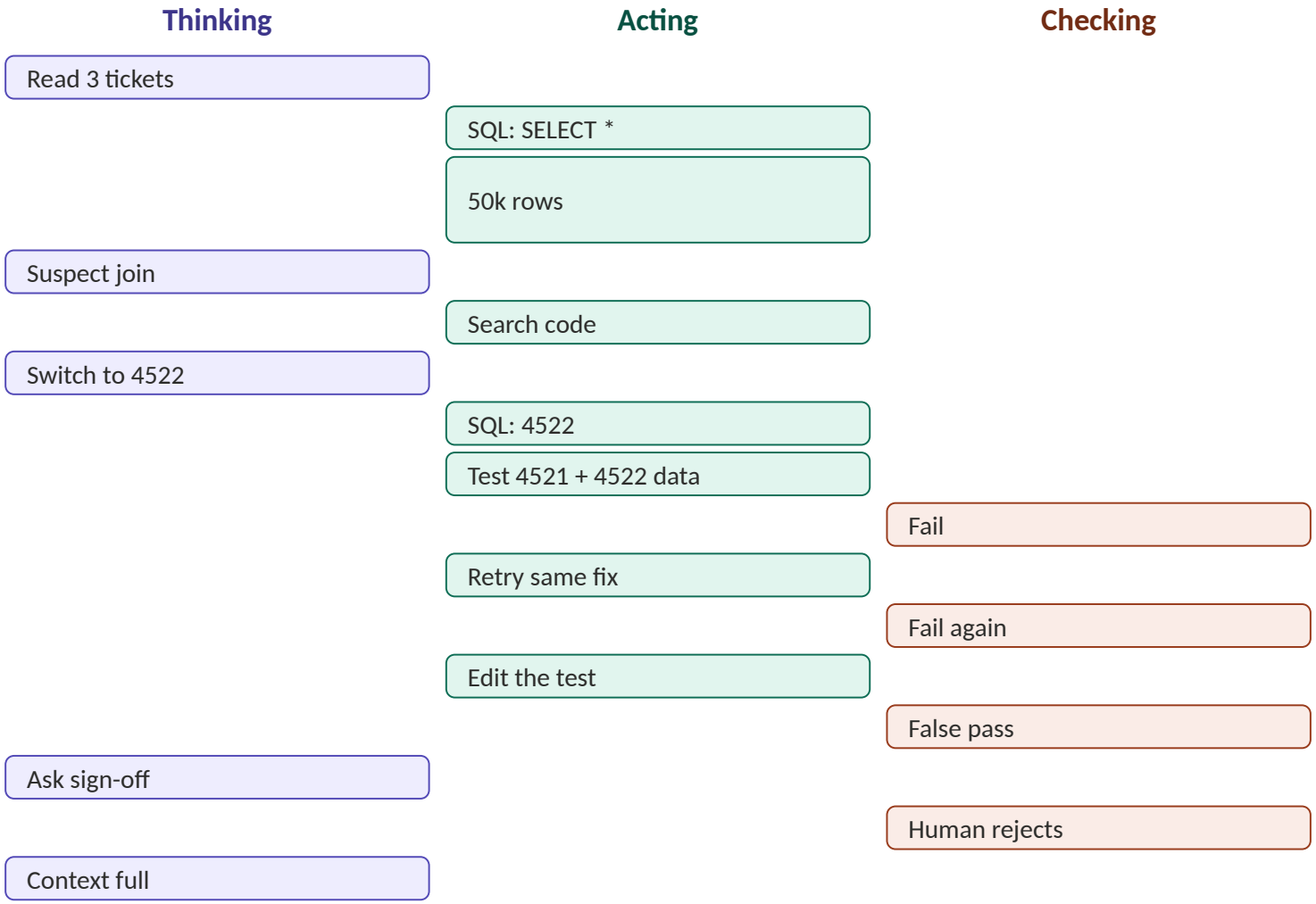
Three jobs in one loop

- **Thinking:** which ticket, what cause, which tool next
- **Acting:** SQL, code search, tests
- **Checking:** did the fix work, does the evidence match

`finish_reason` only signals acting (tool_use) or stopping (end_turn). There is no "plan ready" and no "check passed".

Why Single Agents Fail – Part 2

Thinking, acting and checking are mixed together in one loop. Sort the same messages by job and three separate responsibilities appear.



Three jobs in one loop

- **Thinking:** which ticket, what cause, which tool next
- **Acting:** SQL, code search, tests
- **Checking:** did the fix work, does the evidence match

`finish_reason` only signals acting (tool_use) or stopping (end_turn). There is no "plan ready" and no "check passed".

One loop, three jobs, one exit signal. We'd never let the maker check their own work. Why let an agent?

Failure Modes: What You Observe

Too much work in one loop shows up as recognisable errors, like picking the wrong tool. They fall into three families: thinking, acting and checking.

Thinking failures

Goal drift

See: wanders off the original ask

Cause: long, crowded context

Example: asked to summarise a report, starts rewriting it

Wrong tool

See: right intent, wrong action

Cause: too many similar tools

Example: sends the email instead of drafting it

Acting failures

Infinite loop

See: same step again and again

Cause: no step limit, no memory of failures

Example: keeps re-searching the same flight

Error cascade

See: one bad step spoils the rest

Cause: no isolation or checkpoints

Example: wrong customer ID used in every later step

Checking failures

False success

See: says "done" when it isn't

Cause: grades its own work

Example: "report sent" but the attachment is empty

Premature stop

See: quits halfway, sounds confident

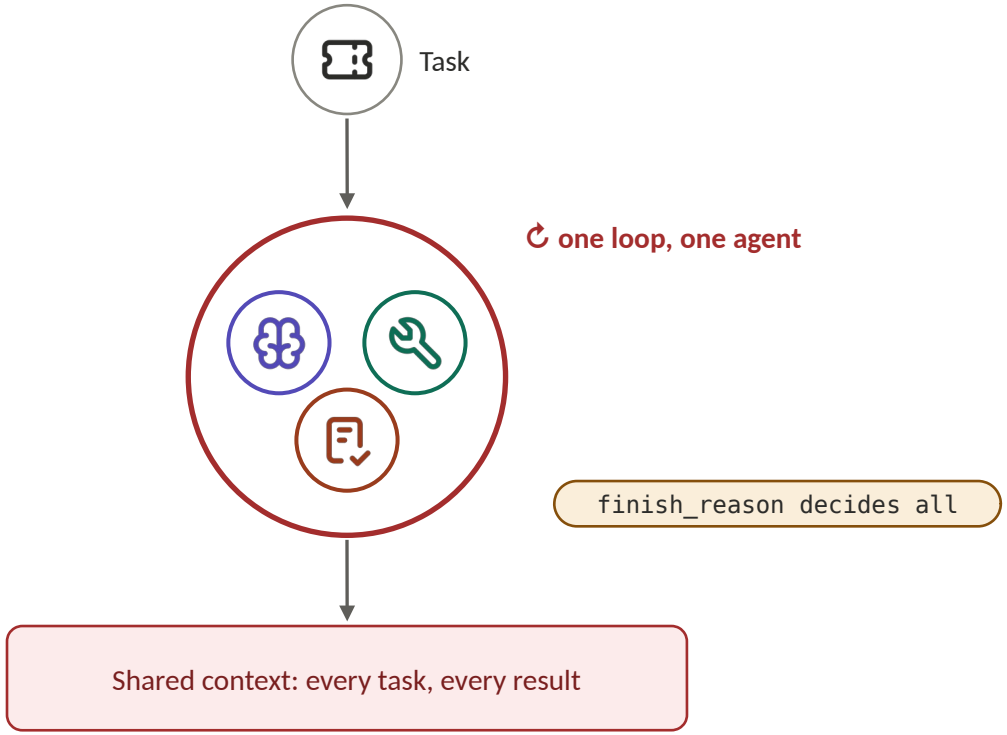
Cause: "stopped talking" read as "finished"

Example: answers 2 of 5 questions

Every failure is one of three jobs going wrong, and every cause is a design choice.

Root Cause: A System Design Problem

Without structure, intelligence degrades. The fix is design, not a bigger model: apply five principles we already trust and the monolith becomes a multi-agent system.



Before

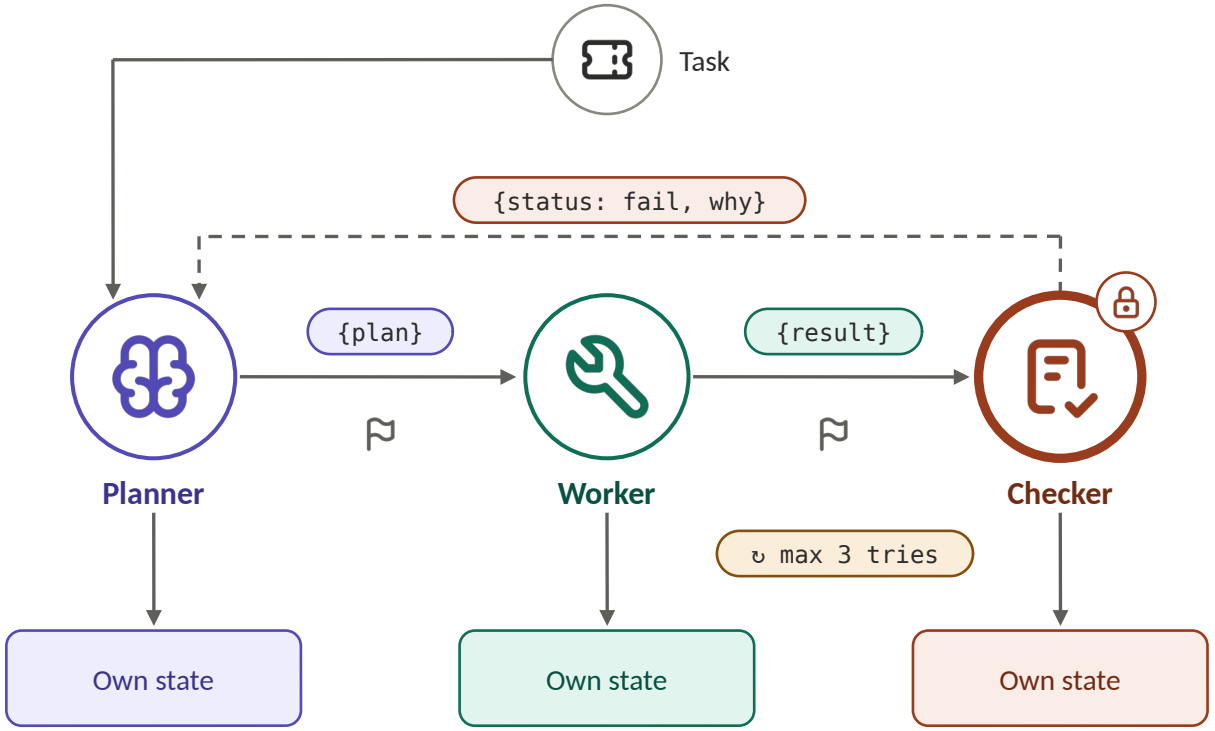
Root cause

One agent does thinking, acting and checking, shares one context, and only stops when finish_reason says so.

- Separation of concerns
- Isolated state
- Bounded execution
- Maker-checker
- Explicit hand-offs

Root Cause: A System Design Problem

Without structure, intelligence degrades. The fix is design, not a bigger model: apply five principles we already trust and the monolith becomes a multi-agent system.



After · all five principles applied

Explicit hand-offs

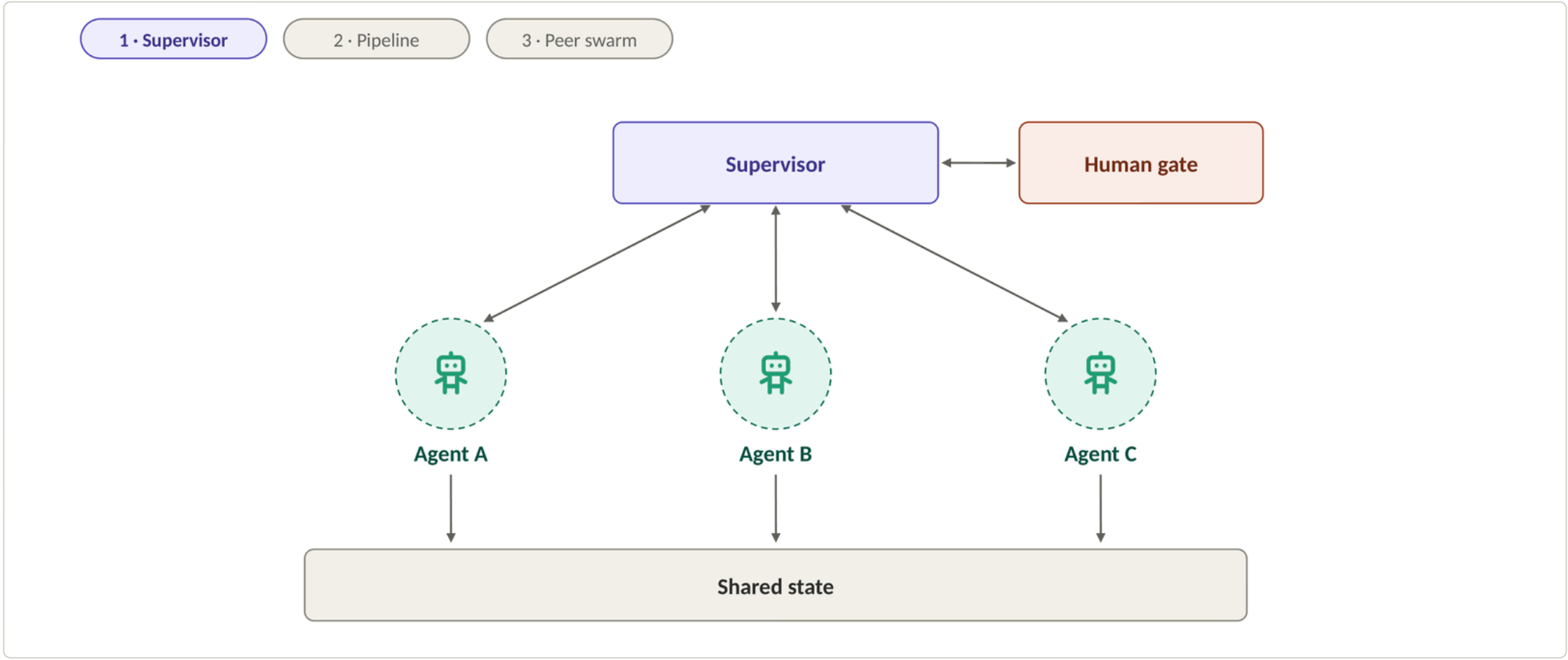
Each step returns a structured status instead of relying on finish_reason. This is a multi-agent system.

- **Separation of concerns**
- **Isolated state**
- **Bounded execution**
- **Maker-checker**
- **Explicit hand-offs**

This isn't an AI problem. It's a system design problem we've already solved, just not for agents yet.

The Core Idea of Multi-Agent Systems

Give each agent one job and connect them with clear hand-offs. The shape can change (supervisor, pipeline, swarm); the building blocks stay the same.



▶ Play animation

A multi-agent system is a well-run team: one job each, clear hand-offs, a shared goal, and someone who signs off.